

Ollama: Running Large Language Models Locally with Ease



In the ever-evolving landscape of artificial intelligence, Ollama is a game-changer for developers and AI enthusiasts working with large language models (LLMs). Unlike cloud-based solutions, Ollama empowers users to run AI models directly on their local machines, offering greater transparency, control, and customization. This article delves into what Ollama is, how it works, its installation process, and how you can seamlessly run models with it.

What is Ollama?

At its core, Ollama is a software platform designed to simplify the process of running open-source LLMs locally. It eliminates the complexities of managing model weights, configurations, and dependencies, allowing users to focus on interacting with the AI rather than dealing with technical roadblocks.

Key Features of Ollama:

- **Local Deployment:** Run LLMs directly on your machine, ensuring greater control over resources and privacy.
- **Open-Source Compatibility:** Works primarily with open-source models, fostering transparency and customization.
- **Simplified Workflow:** Ollama streamlines setup and interaction with LLMs, making it easy to experiment and deploy AI models.
- **Flexible Model Support:** Supports various open-source models like Llama 3, Mistral, Phi-3, Code Llama, and Gemma.

How Ollama Works

Ollama operates by creating a containerized environment for LLMs, ensuring a consistent and isolated setup for each model. These containers encapsulate:

- **Model Weights:** The data that defines an LLM's capabilities.
- **Configuration Files:** Settings that dictate how the model functions.

- **Dependencies:** Software libraries and tools required to run the model smoothly.

By leveraging containerization, Ollama simplifies deployment and eliminates compatibility issues across different systems.

Basic Workflow:

1. **Choose a Model:** Ollama supports a variety of open-source LLMs tailored to different applications.
2. **Configure the Model (Optional):** Users can customize models using a **Modelfile**, specifying version, hardware acceleration, and other parameters.
3. **Run the Model:** Simple CLI commands enable users to create and launch models effortlessly.
4. **Interact with the LLM:** Send prompts to the model and receive responses through a command-line interface or integrations with programming libraries.

Why Run LLMs Locally?

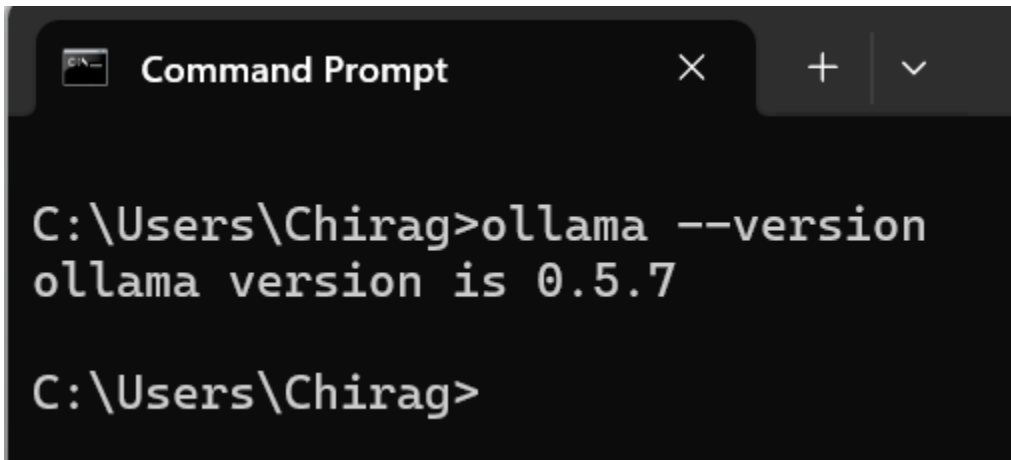
As the demand for LLMs grows, many users seek ways to run them locally for better control and efficiency. Running LLMs on your own machine offers several benefits:

- **Data Privacy:** Keeps sensitive data secure without relying on external servers.
- **Customization:** Modify models to better suit specific applications.
- **Offline Capability:** Once downloaded, models can function without an internet connection.

Installing Ollama

Ollama is compatible with **macOS, Linux, and Windows**. To get started:

1. **Download Ollama:** Visit the [official website](#) and download the version for your OS.
2. **Install Ollama:** Follow the installation instructions for your platform
3. **Verify Installation:** Open a terminal and run: `ollama --version`. If the installation is successful, the version number will be displayed.



```
C:\Users\Chirag>ollama --version
ollama version is 0.5.7

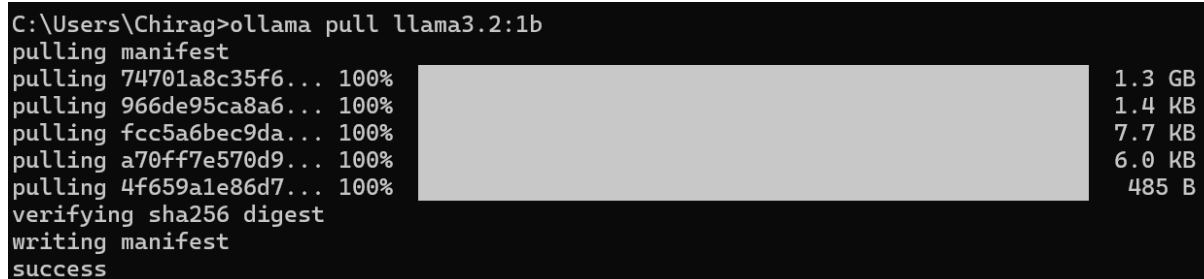
C:\Users\Chirag>
```

Running a Model with Ollama

Once installed, using Ollama to run an LLM is straightforward. Choose the model via the [model list](#) from Ollama

1. Load a Model

Use the CLI to pull your desired model: **`ollama pull llama3.2:1b`**



```
C:\Users\Chirag>ollama pull llama3.2:1b
pulling manifest
pulling 74701a8c35f6... 100%
pulling 966de95ca8a6... 100%
pulling fcc5a6bec9da... 100%
pulling a70ff7e570d9... 100%
pulling 4f659a1e86d7... 100%
verifying sha256 digest
writing manifest
success
```

File	Size
manifest	1.3 GB
74701a8c35f6...	1.4 KB
966de95ca8a6...	7.7 KB
fcc5a6bec9da...	6.0 KB
a70ff7e570d9...	485 B

2. Generate Text

After loading the model, interact with it by using the run command: **ollama run llama3.2:1b**

```
C:\Users\Chirag>ollama run llama3.2:1b
>>> |Send a message (/? for help)
```

Start chatting through the terminal

```
C:\Users\Chirag>ollama run llama3.2:1b
>>> Hi
How can I assist you today?

>>> print hello world in python
You can print "Hello World" in Python by using the 'print()' function. Here's an example:

```python
print("Hello, World!")
```

When you run this code, it will output: `Hello, World!`

>>> |Send a message (/? for help)
```

Running Your First Custom Model

Ollama provides flexibility for users who want to customize their LLMs. Here's how:

1. **Choose a Model:** Select an open-source LLM supported by Ollama.
2. **Create a Modelfile:** Customize your model's configuration by creating a **Modelfile** that defines settings such as version, dependencies, and hardware acceleration.

```
Modelfile X
notebooks > Modelfile
1 # Using existing model with Ollama to set different parameters than the standard one
2 FROM llama3.2:1b
3 # When using a .gguf file FROM ./ollama-model.gguf
4
5 # sets the temperature to 1 [higher is more creative, lower is more coherent]
6 PARAMETER temperature 1
7 # sets the context window size to 4096, this controls how many tokens the LLM can use as context to
8 PARAMETER num_ctx 4096
9
10 # sets a custom system message to specify the behavior of the chat assistant
11 SYSTEM You are a helpful customer support assistant.
```

Code:

```
# Using existing model with Ollama to set different parameters than the
standard one
FROM llama3.2:1b
# When using a .gguf file FROM ./ollama-model.gguf

# sets the temperature to 1 [higher is more creative, lower is more
coherent]
PARAMETER temperature 1
# sets the context window size to 4096, this controls how many tokens the
LLM can use as context to generate the next token
PARAMETER num_ctx 4096

# sets a custom system message to specify the behavior of the chat
assistant
SYSTEM You are a helpful customer support assistant.
```

To add more parameters check [ollama github](#).

3. Create the Model Container:

`ollama create model_name -f path/to/Modelfile`

```
(base) C:\Users\Chirag\Desktop\TMLC\Generative AI course\notebooks>ollama create my_llama3.2:1b -f Modelfile
gathering model components
using existing layer sha256:74701a8c35f6c8d9a4b91f3f3497643001d63e0c7a84e085bed452548fa88d45
using existing layer sha256:966de95ca8a62200913e3f8bfbf84c8494536f1b94b49166851e76644e966396
using existing layer sha256:fcc5a6bec9daf9b561a68827b67ab6088e1dba9d1fa2a50d7bbcc8384e0a265d
using existing layer sha256:a70ff7e570d97baaf4e62ac6e6ad9975e04caa6d900d3742d37698494479e0cd
creating new layer sha256:ad056e502c46275ebc51e8fba1f8464c358e5ece68ddfe48c242425e2961074e
creating new layer sha256:584e6cd273eda8359ff0d82570f61d00f0760a81545b4f7d80f8618a304b2f4f
writing manifest
success
```

4. Run the Model:

`ollama run model_name`

```
(base) C:\Users\Chirag\Desktop\TMLC\Generative AI course\notebooks>ollama run my_llama3.2:1b
>>> Hi
How can I assist you today? Is there something I can help with or would you like to chat about something in
particular?

>>> Send a message (/? for help)
```

5. **Interact with the LLM:** Based on the model's interface, you can send prompts through the CLI or integrate them into your applications.

Python Library Usage:

1. Install ollama using pip -> `pip install ollama`
2. Pull a model using `ollama pull`
3. Check that the model is downloaded using the command `ollama list` which will list all the downloaded models.

```
C:\Users\Chirag>ollama list
NAME                                ID                                SIZE    MODIFIED
my_llama3.2:1b                     73327266b97d                     1.3 GB  About an hour ago
llama3.2:1b                         baf6a787fdff                     1.3 GB  20 hours ago
llama3.2:latest                     a80c4f17acd5                     2.0 GB  6 days ago
granite3-moe:1b                    d84e1e38ee39                     821 MB  2 weeks ago
nomic-embed-text:latest            0a109f422b47                     274 MB  4 weeks ago
customer_support:latest            a13442246316                     2.5 GB  3 months ago
orca-mini:7b-q2_K                  629f61a16f8d                     2.8 GB  3 months ago
bge-m3:latest                      790764642607                     1.2 GB  4 months ago
codellama:latest                   8fdf8f752f6e                     3.8 GB  6 months ago
gemma:2b                           b50d6c999e59                     1.7 GB  6 months ago
```

4. Create a .py file and paste the below code to test in your system.

```
from ollama import chat

response = chat(model='llama3.2:latest', # choose the model from the list
messages=[
    {
        'role': 'user',
        'content': 'Solve  $2x + 5 = 10$ . What is the value of x?',
    },
],
)

print(response['message']['content'])
```

```
(base) C:\Users\Chirag\Desktop\TMLC\Generative AI course\notebooks>python ollama_python.py
To solve for x, we need to isolate x on one side of the equation.
```

```
First, subtract 5 from both sides:
```

$$2x + 5 - 5 = 10 - 5$$

$$2x = 5$$

```
Next, divide both sides by 2:
```

$$(2x) / 2 = 5 / 2$$

$$x = 5/2$$

$$x = 2.5$$

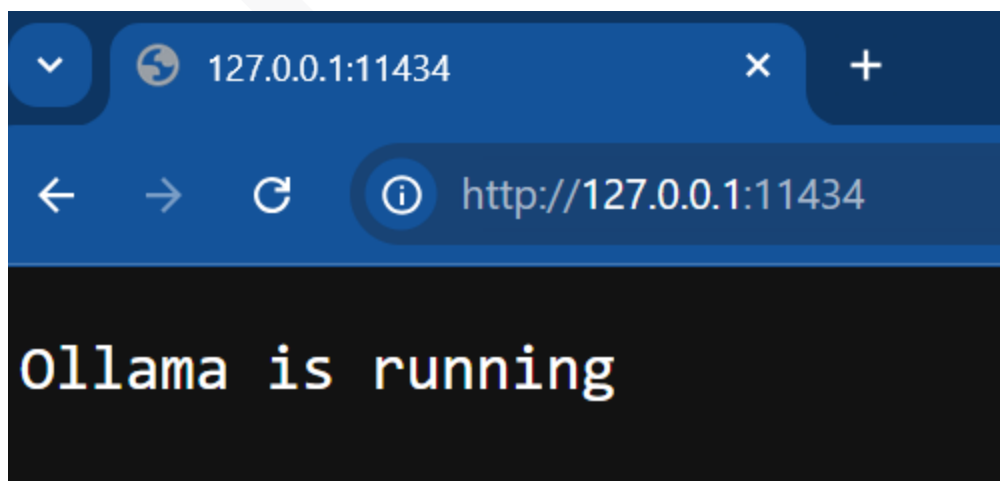
```
So, the value of x is 2.5.
```

Ollama also supports text generation, embedding models and streaming outputs. Visit this [link](#) to check all commands.

Ollama API

The advantage of Ollama API is that any coding language can use the API to use chat, generate or embed functionality of a model given that Ollama is installed on the OS and running.

Ollama runs on the 11434 port of the localhost. To check visit <http://127.0.0.1:11434/> or <http://localhost:11434/>



Example API code

```
import requests
import json

def ollama_embed(prompt):
    url = "http://127.0.0.1:11434/api/embeddings"

    payload = json.dumps({
        "model": "bge-m3:latest",
        "prompt": prompt
    })
    headers = {
        'Content-Type': 'application/json'
    }

    response = requests.request("POST", url, headers=headers, data=payload)

    return response.json()

print(ollama_embed("Hi, what is the standard value of gravity?"))
```

The output should look like:

```
(base) C:\Users\Chirag\Desktop\TMLC\Generative AI course\notebooks>python ollama_api.py
{'embedding': [-2.052567958831787, -0.38057175278663635, -0.9725084900856018, -0.5638558864593506, 0.2515893876552582, -2.632983684539795, 0.35556310415267944, -0.7260065674781799, 0.5883268713951111, -1.126207709312439, -0.734222412109375, 0.8128220438957214, -0.2900846302509308, -0.045496173202991486, 1.4563368558883667, 0.9424336552619934, 0.059671543538570404, -0.7958379983901978, -0.40162673592567444, -0.15740340948104858, -1.2171077728271484, 0.7255825400352478, -0.5115761756896973, 0.06498242914676666, 1.0850017070770264, 1.2810349464416504, -0.9904347658157349, 0.5288900136947632, -0.040611445903778076, 0.4955909252166748, 0.889420211315155, 0.8314581513404846, 0.43261396884918213, -1.1204054355621338, -0.435733675956726
```

Advantages of Using Ollama

Ollama offers numerous benefits for developers and AI enthusiasts:

- **Privacy:** Ensures sensitive data stays on your machine, unlike cloud-based LLM solutions.
- **Customization:** Modify models as needed to suit specific applications.
- **Offline Usage:** No internet dependency once models are downloaded.

- **Ease of Experimentation:** Provides a straightforward way to explore and test different LLMs.

Conclusion

Ollama is revolutionizing the way developers interact with large language models by making them accessible, private, and customizable. Whether you're an AI researcher, developer, or enthusiast, Ollama provides the tools needed to explore and deploy LLMs without the complexity of cloud-based alternatives. With its ease of installation, flexible configuration, and local-first approach, Ollama is a must-have tool for anyone working with AI models.

Ready to dive in? Install Ollama today and start experimenting with powerful LLMs on your machine!